

Fiche de révision — Compilation (S7)

Méthodes clés pour traiter n'importe quel exercice de compilation

1. Les phases d'un compilateur

Phase	Entrée → Sortie	Rôle
Analyse lexicale	code source → suite de tokens	reconnaît les unités lexicales (mots-clés, identifiants, opérateurs) via des automates finis
Analyse syntaxique	tokens → arbre syntaxique (AST)	vérifie que la suite de tokens respecte la grammaire du langage
Analyse sémantique	AST → AST annoté	vérifie types, portées, déclarations (table des symboles)
Génération code intermédiaire	AST → code 3-adresses / IR	traduit vers une représentation indépendante de la machine
Optimisation	IR → IR optimisé	réduit temps/mémoire (propagation de constantes, élimination de code mort...)
Génération de code	IR → code cible/assembleur	allocation de registres, sélection d'instructions

2. Analyse lexicale : méthode type

- **Exercice « expression régulière → automate »** : construire d'abord un NFA (Thompson) puis le déterminer (algorithme des sous-ensembles) pour obtenir un DFA.
- **Exercice « minimiser un DFA »** : partitionner les états en (finaux / non finaux), puis raffiner itérativement les classes tant que deux états d'une même classe ne réagissent pas pareil à une lettre ; fusionner les états indiscernables.
- **Exercice « simuler un automate sur un mot »** : suivre les transitions lettre par lettre ; le mot est accepté si on termine dans un état final (DFA) ou s'il existe un chemin acceptant (NFA).

Construction de Thompson — rappels

a → un seul arc étiqueté a . Concaténation $r_1 r_2$ → on relie l'état final de r_1 à l'état initial de r_2 . Union $r_1 | r_2$ → nouvel état initial avec ϵ -transitions vers r_1 et r_2 . Étoile r^* → ϵ -transitions permettant de boucler zéro ou plusieurs fois.

3. Analyse syntaxique : grammaires et FIRST/FOLLOW

Éliminer la récursivité gauche

Si $A \rightarrow A \alpha \mid \beta$, remplacer par : $A \rightarrow \beta A'$ et $A' \rightarrow \alpha A' \mid \epsilon$.

Factorisation à gauche

Si $A \rightarrow \alpha \beta_1 \mid \alpha \beta_2$, remplacer par $A \rightarrow \alpha A'$ et $A' \rightarrow \beta_1 \mid \beta_2$, pour rendre la grammaire LL(1).

Calcul de FIRST(X)

- Si X est un terminal a : $\text{FIRST}(X) = \{a\}$.
- Si $X \rightarrow \epsilon$ est une production : $\epsilon \in \text{FIRST}(X)$.
- Si $X \rightarrow Y_1 Y_2 \dots Y_n$: ajouter $\text{FIRST}(Y_1) \setminus \{\epsilon\}$; si $\epsilon \in \text{FIRST}(Y_1)$ regarder Y_2 , etc. Si tous les Y_i peuvent dériver ϵ , alors $\epsilon \in \text{FIRST}(X)$.

Calcul de FOLLOW(A)

- $\$$ (fin de chaîne) $\in$ FOLLOW(symbole de départ).
- Si $B \rightarrow \alpha A \beta$: $\text{FIRST}(\beta) \setminus \{\epsilon\} \subseteq \text{FOLLOW}(A)$.
- Si $B \rightarrow \alpha A$ ou $(B \rightarrow \alpha A \beta \text{ avec } \epsilon \in \text{FIRST}(\beta))$: $\text{FOLLOW}(B) \subseteq \text{FOLLOW}(A)$.

Construire la table LL(1)

Pour chaque production $A \rightarrow \alpha$: pour chaque terminal a de $\text{FIRST}(\alpha)$, mettre α dans $\text{Table}[A,a]$. Si $\epsilon \in \text{FIRST}(\alpha)$, pour chaque $b \in \text{FOLLOW}(A)$, mettre α dans $\text{Table}[A,b]$. Deux cases remplies différemment $\Rightarrow$ la grammaire n'est pas LL(1) (conflit).

Analyse descendante (LL) — algorithme de la pile

Pile initialisée avec [$\$, \text{axiome}$]. À chaque étape : si le sommet de pile est un terminal, comparer avec le prochain token d'entrée et dépiler+avancer si égal (sinon erreur) ; si c'est un non-terminal, consulter $\text{Table}[\text{non-terminal}, \text{token courant}]$ et remplacer le sommet par le membre droit de la règle (dans l'ordre inverse).

Analyse ascendante (LR) — idée générale

- On empile des symboles et on décale (shift) le prochain token tant qu'aucune règle complète n'apparaît en haut de pile.
- Quand le sommet de pile correspond au membre droit d'une règle $A \rightarrow \alpha$, on réduit (reduce) : on remplace α par A .
- LR(0)/SLR : construire les items (règles avec un point $\bullet$), calculer les fermetures (closure) et les transitions (goto) pour obtenir l'automate LR ; SLR utilise FOLLOW pour décider des réductions.

4. Analyse sémantique

- **Table des symboles** : associe à chaque identifiant sa portée, son type, son adresse. Gérer les portées imbriquées comme une pile de tables.
- **Vérification de types** : remonter récursivement dans l'AST, typer chaque sous-expression, vérifier la compatibilité (ex. addition entre types compatibles), lever une erreur sinon.
- **Résolution de noms** : chercher un identifiant en partant de la portée la plus interne vers la plus externe.

5. Génération de code intermédiaire

Code à 3 adresses : chaque instruction a au plus un opérateur (ex. $t1 = b * c$; $t2 = a + t1$). Pour un AST, on génère le code fils par fils (parcours postfixe) en créant des variables temporaires.

Exemple rapide : générer le code de $a + b * c$

```
t1 = b * c
t2 = a + t1
```

6. Optimisations courantes (à citer/reconnaître)

- **Propagation de constantes** : remplacer une variable dont la valeur est connue à la compilation par sa valeur.
- **Élimination de code mort** : supprimer le code dont le résultat n'est jamais utilisé.
- **Élimination de sous-expressions communes** : réutiliser le résultat déjà calculé d'une expression identique.
- **Réduction de force** : remplacer une opération coûteuse par une équivalente moins coûteuse (ex. $x*2 \rightarrow x+x$).

7. Pièges classiques en examen

- Oublier ϵ dans FIRST/FOLLOW quand une production peut être vide.
- Confondre NFA et DFA : un NFA peut avoir plusieurs transitions pour une même lettre, pas un DFA.

- Grammaire ambiguë $\neq$ non-LL(1) : ce sont deux notions liées mais distinctes ; toujours vérifier les conflits case par case dans la table.
- Dans LR, bien distinguer conflit shift/reduce et conflit reduce/reduce.